Sesión 25: Persistencia NoSQL / ORM (Implementación de Bases de Datos de Alto Rendimiento con Isar u ObjectBox)

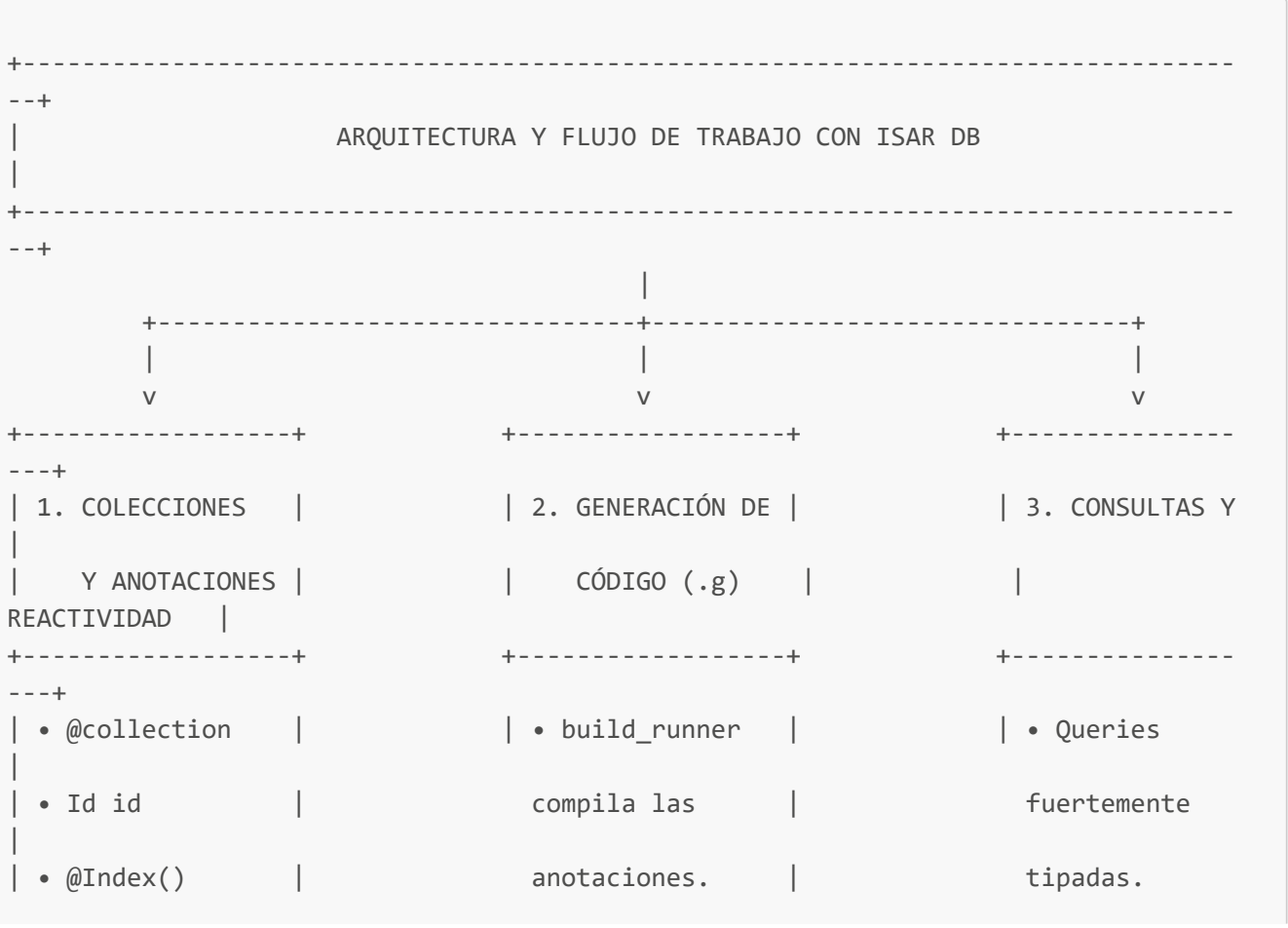
Introducción

Las aplicaciones móviles modernas exigen tiempos de respuesta inmediatos y capacidades de procesamiento de datos eficientes directamente en el dispositivo. Aunque las bases de datos relacionales tradicionales como SQLite (`sqflite`) son efectivas para consultas SQL directas, suelen requerir mapeos manuales intensivos y sobrecostos de serialización. Para escenarios donde se demanda un alto rendimiento, consultas fuertemente tipadas y soporte nativo para modelos de datos orientados a objetos, las bases de datos NoSQL y ORM integradas ofrecen una alternativa superior.

En esta sesión abordaremos la **Persistencia NoSQL / ORM** enfocándonos en **Isar Database**, la base de datos orientada a objetos de ultratransmisión y alto rendimiento diseñada específicamente para Flutter y Dart. Exploraremos el modelado declarativo de colecciones, índices, consultas fuertemente tipadas, transacciones ACID y la integración reactiva con Streams para mantener la interfaz de usuario sincronizada en tiempo real.

Conceptos y Definiciones

Isar se destaca como un motor NoSQL embebido que elimina el mapeo objeto-relacional manual mediante generación de código fuertemente tipada.



```
|
| • @Name()        |             | • Crea esquemas  |             | •
Transacciones  |
| • Enlaces/Links  |               y serializadores |               writeTxn().
|
|   1:N y N:M.     |               optimizados.     |             | • Streams
|
|                  |                en C/C++.       |               watch() en UI.
|
+------------------+             +------------------+             +---------------
---+
```

1. Modelado de Colecciones y Anotaciones `@collection`

- **Colección (`@collection`):** Equivalente a una tabla o documento NoSQL. Es una clase pura de Dart anotada para indicar que debe ser gestionada por Isar.
- **Identificador (`Id`):** Cada colección debe contener un campo identificador entero (`Id id = Isar.autoIncrement;`) que Isar utiliza como clave primaria interna.
- **Índices (`@Index`):** Permiten acelerar drásticamente las búsquedas y ordenamientos sobre atributos específicos o combinaciones de campos (*composite indexes*).
- **Enlaces de Datos (`IsarLink` e `IsarLinks`):** Permite modelar relaciones 1:1, 1:N y N:M de forma orientada a objetos sin necesidad de operaciones `JOIN` manuales.

---

2. Operaciones Transaccionales (ACID)

- **Seguridad Hilo/Aislamiento:** Isar garantiza operaciones de lectura asíncronas no bloqueantes e hilos separados para escrituras.
- **Escrituras Sincrónicas/Asincrónicas (`writeTxn`):** Toda modificación de datos (inserción, edición, borrado) debe ejecutarse dentro de un bloque de transacción explícito:

```
await isar.writeTxn(() async {
  await isar.alumnos.put(nuevoAlumno);
});
```

---

3. Consultas Tipadas y Reactividad (*Watch Streams*)

- **Consultas Declarativas:** Isar genera métodos de filtrado y ordenamiento con sintaxis fluida basados en el sistema de tipos de Dart (`isar.alumnos.filter().promedioGreaterThan(8.0).findAll()`).
- **Sincronización en Tiempo Real (`.watch()`):** Permite escuchar cambios en una consulta específica o en toda la colección retornando un `Stream`, lo que facilita actualizar la interfaz automáticamente mediante un `StreamBuilder`.

Ejemplo Completo con Código Fuente: Sistema NoSQL de Control Escolar en ESCOM

El siguiente ejemplo simula el uso de una base de datos NoSQL de alto rendimiento con Isar para gestionar una lista de proyectos escolares, permitiendo la búsqueda reactiva y filtrado en tiempo real.

> **Configuración del proyecto (`pubspec.yaml`):**
>
> ```
> dependencies:
>   flutter:
>     sdk: flutter
>   isar: ^3.1.0+1
>   isar_flutter_libs: ^3.1.0+1
>
> dev_dependencies:
>   build_runner: ^2.4.9
>   isar_generator: ^3.1.0+1
> ```
>
> *Comando para generación de código:* `dart run build_runner build --delete-conflicting-outputs`

Código Fuente Explicado (`lib/main.dart`)

```
import 'package:flutter/material.dart';

void main() {
  runApp(const DemoIsarNoSqlApp());
}

// ============================================================================
// DEFINICIÓN DECLARATIVA DE LA COLECCIÓN ISAR
// ============================================================================
// En un entorno de producción con generación de código, la colección se declara
así:
//
// import 'package:isar/isar.dart';
// part 'proyecto.g.dart';
//
// @collection
// class ProyectoEscolar {
//   Id id = Isar.autoIncrement;
//
//   @Index(type: IndexType.value)
//   late String titulo;
//
//   late String laboratorio;
//   late double calificacion;
//   late bool entregado;
// }
```

```dart
/// Simulación estructural de la entidad persistente NoSQL
class ProyectoEscolar {
  int id;
  String titulo;
  String laboratorio;
  double calificacion;
  bool entregado;

  ProyectoEscolar({
    required this.id,
    required this.titulo,
    required this.laboratorio,
    required this.calificacion,
    required this.entregado,
  });

  ProyectoEscolar copyWith({
    int? id,
    String? titulo,
    String? laboratorio,
    double? calificacion,
    bool? entregado,
  }) {
    return ProyectoEscolar(
      id: id ?? this.id,
      titulo: titulo ?? this.titulo,
      laboratorio: laboratorio ?? this.laboratorio,
      calificacion: calificacion ?? this.calificacion,
      entregado: entregado ?? this.entregado,
    );
  }
}

// ============================================================================
// GESTOR DE BASE DE DATOS NOSQL (SIMULACIÓN DE MOTOR ISAR REACTIVO)
// ============================================================================

class IsarDatabaseManager {
  static final IsarDatabaseManager instance = IsarDatabaseManager._init();
  IsarDatabaseManager._init();

  final List<ProyectoEscolar> _storage = [
    ProyectoEscolar(
      id: 1,
      titulo: 'Sistema de Visión por Computadora',
      laboratorio: 'Visión Artificial - ESCOM',
      calificacion: 9.5,
      entregado: true,
    ),
    ProyectoEscolar(
      id: 2,
      titulo: 'Agente Inteligente con Llama 3',
      laboratorio: 'Ciberseguridad & IA',
```

```
      calificacion: 10.0,
      entregado: true,
    ),
  ];

  // Simulación de transacción de escritura NoSQL (writeTxn)
  Future<void> guardarProyecto(ProyectoEscolar proyecto) async {
    await Future.delayed(const Duration(milliseconds: 150)); // Latencia ultra-
baja
    if (proyecto.id == 0) {
      proyecto.id = DateTime.now().millisecondsSinceEpoch;
      _storage.add(proyecto);
    } else {
      final index = _storage.indexWhere((p) => p.id == proyecto.id);
      if (index != -1) {
        _storage[index] = proyecto;
      }
    }
  }

  Future<void> eliminarProyecto(int id) async {
    await Future.delayed(const Duration(milliseconds: 100));
    _storage.removeWhere((p) => p.id == id);
  }

  // Consulta NoSQL filtrada
  Future<List<ProyectoEscolar>> consultarProyectos({String filtro = ''}) async {
    await Future.delayed(const Duration(milliseconds: 100));
    if (filtro.isEmpty) return List.from(_storage);
    return _storage
        .where((p) =>
            p.titulo.toLowerCase().contains(filtro.toLowerCase()) ||
            p.laboratorio.toLowerCase().contains(filtro.toLowerCase()))
        .toList();
  }
}

// ============================================================================
// INTERFAZ DE USUARIO (PRESENTATION LAYER)
// ============================================================================

class DemoIsarNoSqlApp extends StatelessWidget {
  const DemoIsarNoSqlApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'ESCOM - NoSQL / ORM con Isar',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        useMaterial3: true,
        colorScheme: ColorScheme.fromSeed(
          seedColor: const Color(0xFF6B112D), // Guinda Institucional
        ),
```

```
      ),
      home: const PantallaGestionProyectos(),
    );
  }
}

class PantallaGestionProyectos extends StatefulWidget {
  const PantallaGestionProyectos({super.key});

  @override
  State<PantallaGestionProyectos> createState() =>
      _PantallaGestionProyectosState();
}

class _PantallaGestionProyectosState extends State<PantallaGestionProyectos> {
  final _tituloController = TextEditingController();
  final _labController = TextEditingController();
  final _califController = TextEditingController();
  final _busquedaController = TextEditingController();

  List<ProyectoEscolar> _proyectos = [];
  bool _cargando = false;

  @override
  void initState() {
    super.initState();
    _actualizarLista();
  }

  Future<void> _actualizarLista() async {
    setState(() => _cargando = true);
    final resultados = await IsarDatabaseManager.instance.consultarProyectos(
      filtro: _busquedaController.text,
    );
    setState(() {
      _proyectos = resultados;
      _cargando = false;
    });
  }

  Future<void> _agregarProyecto() async {
    if (_tituloController.text.isEmpty || _labController.text.isEmpty) return;

    final nuevo = ProyectoEscolar(
      id: 0,
      titulo: _tituloController.text,
      laboratorio: _labController.text,
      calificacion: double.tryParse(_califController.text) ?? 8.0,
      entregado: true,
    );

    // Ejecución de escritura NoSQL
    await IsarDatabaseManager.instance.guardarProyecto(nuevo);
```

```
    _tituloController.clear();
    _labController.clear();
    _califController.clear();

    await _actualizarLista();
  }

  Future<void> _eliminar(int id) async {
    await IsarDatabaseManager.instance.eliminarProyecto(id);
    await _actualizarLista();
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Persistencia NoSQL / ORM (Isar)'),
        backgroundColor: Theme.of(context).colorScheme.primary,
        foregroundColor: Colors.white,
      ),
      body: Padding(
        padding: const EdgeInsets.all(16.0),
        child: Column(
          children: [
            // BUSCADOR EN TIEMPO REAL
            TextField(
              controller: _busquedaController,
              onChanged: (_) => _actualizarLista(),
              decoration: const InputDecoration(
                labelText: 'Buscar por Título o Laboratorio (Consulta Isar)',
                prefixIcon: Icon(Icons.search),
                border: OutlineInputBorder(),
              ),
            ),
            const SizedBox(height: 16),

            // FORMULARIO DE REGISTRO
            Card(
              elevation: 2,
              child: Padding(
                padding: const EdgeInsets.all(12.0),
                child: Column(
                  crossAxisAlignment: CrossAxisAlignment.stretch,
                  children: [
                    const Text(
                      'Nuevo Proyecto NoSQL',
                      style: TextStyle(fontWeight: FontWeight.bold),
                    ),
                    const SizedBox(height: 8),
                    TextField(
                      controller: _tituloController,
                      decoration: const InputDecoration(
                        labelText: 'Título del Proyecto',
                        isDense: true,
```

```
                    ),
                  ),
                  const SizedBox(height: 6),
                  TextField(
                    controller: _labController,
                    decoration: const InputDecoration(
                      labelText: 'Laboratorio / Academia',
                      isDense: true,
                    ),
                  ),
                  const SizedBox(height: 6),
                  TextField(
                    controller: _califController,
                    keyboardType: TextInputType.number,
                    decoration: const InputDecoration(
                      labelText: 'Calificación Inicial',
                      isDense: true,
                    ),
                  ),
                  const SizedBox(height: 10),
                  ElevatedButton.icon(
                    onPressed: _agregarProyecto,
                    icon: const Icon(Icons.add_task),
                    label: const Text('Insertar en Isar DB (writeTxn)'),
                  ),
                ],
              ),
            ),
          ),
          const SizedBox(height: 16),

          // LISTADO DE PROYECTOS
          Expanded(
            child: _cargando
                ? const Center(child: CircularProgressIndicator())
                : _proyectos.isEmpty
                    ? const Center(child: Text('Sin registros NoSQL
encontrados.'))
                    : ListView.builder(
                        itemCount: _proyectos.length,
                        itemBuilder: (context, index) {
                          final proyecto = _proyectos[index];
                          return Card(
                            margin: const EdgeInsets.only(bottom: 8),
                            child: ListTile(
                              leading: CircleAvatar(
                                backgroundColor: proyecto.calificacion >= 9.0
                                    ? Colors.green.shade100
                                    : Colors.amber.shade100,
                                child: Text(
                                  '${proyecto.calificacion}',
                                  style: const TextStyle(
                                    fontSize: 12,
                                    fontWeight: FontWeight.bold,
```

```
                                    ),
                                  ),
                                ),
                                title: Text(proyecto.titulo),
                                subtitle: Text(proyecto.laboratorio),
                                trailing: IconButton(
                                  icon: const Icon(Icons.delete_outline,
                                      color: Colors.red),
                                  onPressed: () => _eliminar(proyecto.id),
                                ),
                              ),
                            );
                          },
                        ),
                  ),
                ],
              ),
            ),
          );
        }
      }
```

Prácticas Guiadas / Laboratorio

Práctica 1: Definición de Enlaces 1:N con `IsarLinks`

1. Crea una segunda colección llamada `Profesor` con sus atributos base e identificador `Id id`.
2. Define dentro de la colección `ProyectoEscolar` un enlace `final profesores = IsarLinks<Profesor>();`.
3. Implementa en el laboratorio la inserción relacional de un profesor asignado al proyecto utilizando la transacción `await proyecto.profesores.save()`.

Práctica 2: Consultas Reactivas con Streams (`.watch()`)

1. Implementa una consulta reactiva sobre la colección Isar que escuche cualquier cambio en los proyectos:

```
Stream<List<ProyectoEscolar>> escucharProyectos() {
  return isar.proyectoEscolars.where().watch(fireImmediately: true);
}
```

2. Sustituye la llamada manual de actualización en la interfaz de usuario por un widget `StreamBuilder<List<ProyectoEscolar>>` para comprobar cómo la UI se redibuja automáticamente ante cualquier inserción o borrado.

Resumen y Puntos Clave

- **Rendimiento Orientado a Objetos:** Isar y ObjectBox ofrecen velocidades de lectura/escritura muy superiores a SQLite al evitar el parseo intermedio de cadenas SQL y mapas dinámicos.
- **Consultas Fuertemente Tipadas:** Las búsquedas se escriben directamente en sintaxis de Dart, detectando errores de consulta en tiempo de compilación.
- **Transacciones Obligatorias (`writeTxn`):** Garantizan la integridad ACID de la base de datos NoSQL; las operaciones de lectura son asíncronas e hiperrápidas.
- **Reactividad Integrada:** La capacidad de observar consultas (`.watch()`) simplifica la sincronización en tiempo real entre la capa de persistencia y la interfaz visual.

Bibliografía y Referencias

1. **Simon Leier.** (2024). *Isar Database: Fast Cross-Platform Database for Flutter*. Recuperado de: https://isar.dev
2. **ObjectBox Ltd.** (2024). *ObjectBox Flutter: Super-fast database for Flutter & Dart*. Recuperado de: https://objectbox.io/flutter/
3. **Moedano, B.** (2023). *Desarrollo de Aplicaciones Móviles Adaptativas con Flutter*. Editorial Alfaomega. ISBN: 978-6075389102.